



Chapter 3: Introduction to SQL

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Outline

- Overview of The SQL Query Language
- SQL Data Definition
- Basic Query Structure of SQL Queries
- Additional Basic Operations
- Set Operations
- Null Values
- Aggregate Functions
- Nested Subqueries
- Modification of the Database



History

- IBM **Sequel language** developed as part of **System R** project at the IBM San Jose Research Laboratory
- Renamed Structured Query Language (SQL)
- ANSI and ISO standard SQL:
 - **SQL-86 –Official Standard**
 - **SQL-89 –Small improvements**
 - **SQL-92 –Major updates**
 - **SQL:1999 –New features**
 - **SQL:2003-Further addition**
- Commercial systems offer most, if not all, **SQL-92 features**, plus varying feature sets from later standards and special proprietary features.
 - Not all **examples here may work on your particular system.**



SQL Parts

■ **DML** -- provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.

- **SELECT * FROM Student;**
- **INSERT INTO Student VALUES(101,'Ravi',20);**
- **UPDATE Student SET age=21 WHERE id=101;**
- **DELETE FROM Student WHERE id=101;**

■ **integrity** – = enforce correctness

the DDL includes commands for specifying integrity constraints. Integrity constraints prevent wrong data from entering the database

- **CREATE TABLE Student(**
- **id INT PRIMARY KEY,**
- **name VARCHAR(30) NOT NULL,**
- **age INT CHECK(age >= 18)**
- **);**



SQL Parts

- **View definition** -- virtual tables for security/simplification

The DDL includes commands for defining views. A view does not store data separately. It stores the query.

- **CREATE VIEW CS_Students AS**

SELECT id, name FROM Student WHERE dept='CSE';

- **Transaction control** –includes commands for specifying the beginning and ending of transactions.

Used to maintain **consistency** when multiple operations happen together.

- **START TRANSACTION;**
- **UPDATE Account SET balance = balance - 500 WHERE accno=1;**
- **UPDATE Account SET balance = balance + 500 WHERE accno=2;**
- **COMMIT;**

--money should not be deducted without being credited



SQL Parts

- **Embedded SQL and dynamic SQL** -- define how SQL statements can be embedded within general-purpose programming languages.
- SQL written inside programming languages like C/Java/Python.

- **Java program using JDBC**
- **Python using MySQL connector**

Dynamic SQL- Dynamic SQL means SQL query is built during execution

- **query = "SELECT * FROM Student WHERE dept = " + dept + """**
- **Authorization** – includes commands for specifying access rights to relations and views.
- Defines **who can access what**.
 - **GRANT SELECT ON Student TO user1;**
 - **REVOKE SELECT ON Student FROM user1;**



Data Definition Language

The SQL data-definition language (DDL) allows the specification of information about relations, including:

- The **schema** for each relation.
- The **type of values** associated with each attribute.
- The **Integrity constraints**
- The set of indices to be maintained for each relation **for faster searching**
- **Security and authorization** information for each relation.-DDL can define which users can view or modify a table.
- The **physical storage structure** of each relation on disk. how the table data is actually saved

```
CREATE TABLE Student (  
  id INT PRIMARY KEY,  
  age INT CHECK(age >= 18),  
  dept VARCHAR(10) NOT NULL);
```

```
CREATE INDEX idx_dept ON Student(dept);
```

```
GRANT SELECT ON Student TO user1;  
REVOKE INSERT ON Student FROM user1;  
ENGINE=InnoDB;  
TABLESPACE users;
```

```
PARTITION BY RANGE(adm_year) (  
  PARTITION p2024 VALUES LESS THAN  
  (2025),  
  PARTITION p2025 VALUES LESS THAN  
  (2026)  
);
```

```
SELECT * FROM Student WHERE adm_year  
= 2025;
```



Domain Types in SQL

- Datatype decides **what kind of values are allowed and how much memory is used.**
- **char(*n*).** Fixed **length character string**, with user-specified **length *n***.
 - CHAR(1) → M/F
- **varchar(*n*).** Variable length character strings, with user-specified maximum length *n*.
 - name VARCHAR(50)
- **int.** Integer (a finite subset of the integers that is machine-dependent).
- **smallint.** **Small integer** (a machine-dependent subset of the integer domain type).
- **numeric(*p*,*d*).** Fixed point number, with user-specified precision of *p* digits, with *d* digits to the right of decimal point. (ex., **numeric(3,1)**, allows **44.5 to be stores exactly, but not 444.5 or 0.32**)
 - CGPA NUMERIC(3,2) → 9.25, 8.50
- **real, double precision.** Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(*n*).** Floating point number, with user-specified precision of at least *n* digits. ---approximate (science data, emperature, distance, speed)



Create Table Construct

- An SQL relation is defined using the **create table** command:

create table *r*

$(A_1 D_1, A_2 D_2, \dots, A_n D_n,$
 (integrity-constraint₁),
 ...,
 (integrity-constraint_k))

- *r* is the name of the relation
 - each A_i is an attribute name in the schema of relation *r*
 - D_i is the data type of values in the domain of attribute A_i
- Example:

```
create table instructor (  
    ID           char(5),  
    name        varchar(20),  
    dept_name   varchar(20),  
    salary     numeric(8,2))
```



Integrity Constraints in Create Table

- Types of integrity constraints
 - **primary key** ($A_1, \dots, A_n$)
 - **foreign key** ($A_m, \dots, A_n$) **references** r
 - **not null**
- SQL prevents any update to the database that violates an integrity constraint.
- Example:

```
create table instructor (  
    ID          char(5),  
    name       varchar(20) not null,  
    dept_name varchar(20),  
    salary    numeric(8,2),  
    primary key (ID),  
    foreign key (dept_name) references department);
```



And a Few More Relation Definitions

- **create table** *student* (
 ID **varchar**(5),
 name **varchar**(20) not null,
 dept_name **varchar**(20),
 tot_cred **numeric**(3,0),
 primary key (*ID*),
 foreign key (*dept_name*) **references** *department*);
- **create table** *takes* (
 ID **varchar**(5),
 course_id **varchar**(8),
 sec_id **varchar**(8),
 semester **varchar**(6),
 year **numeric**(4,0),
 grade **varchar**(2),
 primary key (*ID*, *course_id*, *sec_id*, *semester*, *year*) ,
 foreign key (*ID*) **references** *student*,
 foreign key (*course_id*, *sec_id*, *semester*, *year*) **references** *section*);



And more still

- **create table** *course* (
 course_id **varchar**(8),
 title **varchar**(50),
 dept_name **varchar**(20),
 credits **numeric**(2,0),
 primary key (*course_id*),
 foreign key (*dept_name*) **references**
 department);



Updates to tables

- **Insert**

- insert into *instructor* values ('10211', 'Smith', 'Biology', 66000);

- **Delete**

- Remove all tuples from the *student* relation
 - **delete from *student***

- **Drop Table**

- **drop table *r***

- **Alter**

- **alter table *r* add *A D***
 - where *A* is the name of the attribute to be added to relation *r* and *D* is the domain of *A*.
 - All existing tuples in the relation are assigned *null* as the value for the new attribute.
- **alter table *r* drop *A***
 - where *A* is the name of an attribute of relation *r*
 - Dropping of attributes not supported by many databases.



Basic Query Structure

- A typical SQL query has the form:

select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P

- A_i represents an attribute
 - r_i represents a relation
 - P is a predicate.
- The result of an SQL query is a relation.



The select Clause

- The **select** clause lists the attributes desired in the result of a query
 - corresponds to the projection operation of the relational algebra
- Example: find the names of all instructors:
select name
from instructor
- NOTE: SQL names are case insensitive (i.e., you may use upper- or lower-case letters.)
 - E.g., *Name* $\equiv$ *NAME* $\equiv$ *name*-o all these identifiers refer to the same column
 - Some people use upper case wherever we use bold font.



The select Clause (Cont.)

- SQL allows duplicates in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **distinct** after select.
- Find the department names of all instructors, and remove duplicates

```
select distinct dept_name  
from instructor
```

- The keyword **all** specifies that duplicates should not be removed.

```
select all dept_name  
from instructor
```

<i>dept_name</i>
Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.



The select Clause (Cont.)

- An asterisk in the select clause denotes “all attributes”

```
select *  
from instructor
```

- An attribute can be a literal with no **from** clause

```
select '437'
```

- Results is a table with **one column and a single row with value “437”**
- Can give the **column a name using:**

```
select '437' as FOO
```

- An attribute can be a literal with **from** clause

```
select 'A'  
from instructor
```

- Result is a table with one column and N rows (number of tuples in the *instructors* table), each row with value “A”



The select Clause (Cont.)

- The **select** clause can contain arithmetic expressions involving the operation, +, −, *, and /, and operating on constants or attributes of tuples. **calculate new values** using operators
 - The query:

select *ID, name, salary/12*
from *instructor*

would return a relation that is the same as the *instructor* relation, except that the value of the attribute *salary* is divided by 12.

- Can rename “salary/12” using the **as** clause:

select *ID, name, salary/12 as monthly_salary*
SELECT *ID, name, salary*1.10 AS increased_salary*
FROM *instructor;*



The where Clause

- The **where** clause specifies conditions that the result must satisfy
 - Corresponds to the selection predicate of the relational algebra.
- To find all instructors in Comp. Sci. dept

```
select name  
from instructor  
where dept_name = 'Comp. Sci.'
```

- SQL allows the use of the logical connectives **and**, **or**, and **not**
- The operands of the logical connectives can be expressions involving the comparison operators **<**, **<=**, **>**, **>=**, **=**, and **<>**.
- Comparisons can be applied to results of arithmetic expressions
- To find all instructors in Comp. Sci. dept with salary > 70000

```
select name  
from instructor  
where dept_name = 'Comp. Sci.' and salary > 70000
```

<i>name</i>
Katz
Brandt



The from Clause

- The **from** clause lists the relations involved in the query
 - **Corresponds to the Cartesian product operation of the relational algebra.**
- Find the Cartesian product *instructor* X *teaches*

```
select *  
from instructor, teaches
```

- **generates every possible instructor – teaches pair, with all attributes from both relations.**
 - **For common attributes (e.g., *ID*), the attributes in the resulting table are renamed using the relation name (e.g., *instructor.ID*)**
- Cartesian product not very useful directly, but useful combined with where-clause condition (selection operation in relational algebra).



Examples

- Find the names of all instructors who have taught some course and the course_id
 - **select** *name, course_id*
from *instructor , teaches*
where *instructor.ID = teaches.ID*
- Find the names of all instructors in the Art department who have taught some course and the course_id
 - **select** *name, course_id*
from *instructor , teaches*
where *instructor.ID = teaches.ID*
and *instructor.dept_name = 'Art'*

<i>name</i>	<i>course_id</i>
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181



The Rename Operation

- The SQL allows renaming relations and attributes using the **as** clause:

old-name **as** *new-name*

- Find the names of all instructors who have a higher salary than some instructor in 'Comp. Sci'.
 - **select distinct** *T.name*
from *instructor* **as** *T*, *instructor* **as** *S*
where *T.salary* > *S.salary* **and** *S.dept_name* = 'Comp. Sci.'
- Keyword **as** is optional and may be omitted
instructor **as** *T* $\equiv$ *instructor T*



Self Join Example

- **Self Join means joining a table with itself**
Because the table contains relationships between rows of the *same entity*.
- Relation *emp-super*

<i>person</i>	<i>supervisor</i>
Bob	Alice
Mary	Susan
Alice	David
David	Mary

- Find the supervisor of “Bob”
- Find the supervisor of the supervisor of “Bob”
- o the supervisor is also a **person** who appears in the same table.
- Can you find ALL the supervisors (direct and indirect) of “Bob”?

```
SELECT e2.supervisor  
FROM emp_super e1,  
emp_super e2
```

```
WHERE e1.person = 'Bob'  
AND e1.supervisor =  
e2.person;
```



String Operations

- SQL includes a string-matching operator for comparisons on character strings. **-pattern matching**
- **LIKE helps SQL find text patterns instead of exact words**
- The operator **like** uses patterns that are described using two special characters:
 - **percent (%). The % character matches any substring.**
 - **underscore (_). The _ character matches any character.**
- **Find the names of all instructors whose name includes the substring “dar”.**

Match the string “100%”

like '100 \% ' escape '\'

Escape character is like telling SQL: ‘Hey, treat the next character normally

in that above we use backslash (\) as the escape character.



String Operations (Cont.)

- **Patterns are case sensitive.**
- Pattern matching examples:
 - 'Intro%' matches any string beginning with "Intro".
 - '%Comp%' matches any string containing "Comp" as a substring.
 - **'___' matches any string of exactly three characters** → Exactly 3 characters
 - Find instructor names that have **exactly 4 characters**.
 - **'___%' matches any string of at least three characters.->** At least 3 characters
 - Find instructor names that have **at least 3 characters**.
 - Who has a name containing 'an'?"
 - What does this match?
 - LIKE '_a%'



- Find the names of all instructors whose name includes the substring “dar”.

```
select name
from instructor
where name like '%dar%' → Any name that has dar inside it
```

Find instructor names that have **exactly 4 characters**.

- **SELECT name**
- **FROM instructor**
- **WHERE name LIKE '____';**

Find instructor names that have **at least 3 characters**.

- **SELECT name**
- **FROM instructor**
- **WHERE name LIKE '___%';**

Who has a name containing ‘an’?”

- **SELECT name**
- **FROM instructor**
- **WHERE name LIKE '%an%';**



String Operations (Cont.)

- SQL supports a variety of string operations such as
 - concatenation (using "||")--→for **string concatenation**.
 - **SELECT first_name || ' ' || last_name AS full_name**
 - **FROM instructor;**
 - converting from upper to lower case (and vice versa)
 - **UPPER(name), LOWER(name)**
 - finding string length, extracting substrings, etc.
 - **LENGTH(name)**
 - Find the length of each instructor's name.
 - **SELECT name, LENGTH(name) AS name_length**
 - **FROM instructor;**



Ordering the Display of Tuples

- Databases also arrange data using **ORDER BY**.
- List in alphabetic order the names of all instructors

select distinct *name*
from *instructor*
order by *name* → Ascending (A–Z) is the default

- We may specify **desc** for descending order or **asc** for ascending order, for each attribute; ascending order is the default.
 - Example: **order by *name* desc**
- Can sort on multiple attributes
 - Example: **order by *dept_name*, *name***
 - SELECT *name*, *dept_name*
 - FROM *instructor*
 - ORDER BY *dept_name* ASC, *name* DESC;



Where Clause Predicates

- SQL includes a **between** comparison operator
- Example: Find the names of all instructors with salary between \$90,000 and \$100,000
- **select name**
from instructor
where salary between 90000 and 100000
 - Same as $\rightarrow$ salary $\geq$ 90000 AND salary $\leq$ 100000
- **Tuple comparison-tuple comparison** means comparing a **pair (two values together)** at once.
 - **select name, course_id**
from instructor, teaches
where (instructor.ID, dept_name) = (teaches.ID, 'Biology');
 - So tuple comparison is simply a **shortcut** for writing two conditions.



Matching two columns together

- **For teaches ID = 101:**
- Compare:
- instructor tuple = (101, Biology)
- teaches tuple = (101, Biology) ---MATCH
- → Output: Ravi, BIO-101

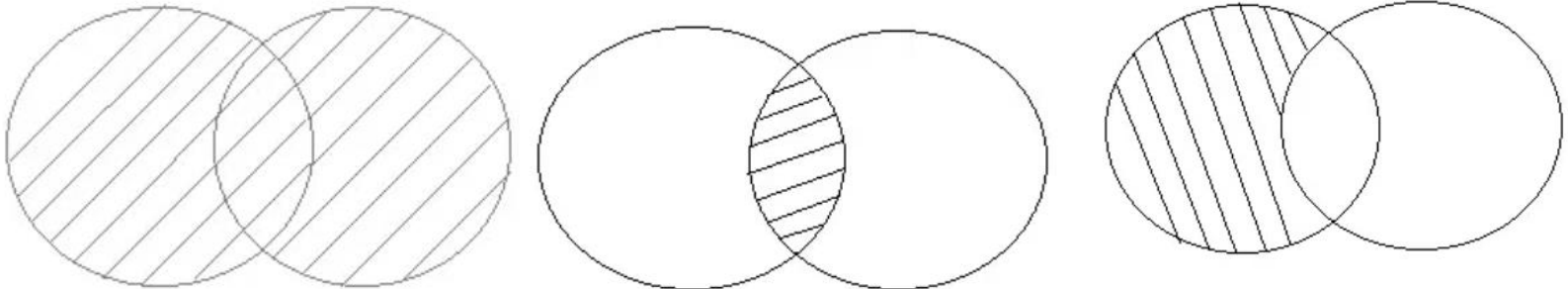
- **For teaches ID = 102:**
- Compare:
- instructor tuple = (102, Comp. Sci)
- teaches tuple = (102, Biology) --- NOT MATCH
- → Not included

```
select name, course_id  
from instructor, teaches  
where instructor.ID =  
teaches.ID  
and dept_name =  
'Biology';
```



Set Operations (Cont.)

- Set operations **union**, **intersect**, and **except**
 - Each of the above operations automatically eliminates duplicates
 - **UNION**: Merges all unique rows from two or more SELECT statements, eliminating duplicates.
 - **SELECT** *column_name(s)* **FROM** *table1*
UNION
SELECT *column_name(s)* **FROM** *table2*;
 - **INTERSECT**: Returns only the rows that appear in both SELECT statements.
 - **EXCEPT**: Returns rows from the first SELECT statement that do not appear in the second.





Set Operations

- **Find courses that ran in Fall 2017 or in Spring 2018**

(select *course_id* from *section* where *sem* = 'Fall' and *year* = 2017)

union

(select *course_id* from *section* where *sem* = 'Spring' and *year* = 2018)

- **Find courses that ran in Fall 2017 and in Spring 2018**

(select *course_id* from *section* where *sem* = 'Fall' and *year* = 2017)

intersect

(select *course_id* from *section* where *sem* = 'Spring' and *year* = 2018)

- **Find courses that ran in Fall 2017 but not in Spring 2018**

(select *course_id* from *section* where *sem* = 'Fall' and *year* = 2017)

except

(select *course_id* from *section* where *sem* = 'Spring' and *year* = 2018)



- **Fall 2017 courses (A):**
 $\{ \text{CS101, CS102, MA101, CS101} \}$
Spring 2018 courses (B):
 $\{ \text{CS102, EC201, MA101, MA101} \}$
- $A \cup B = \{ \text{CS101, CS102, MA101, EC201} \}$
- $A \cap B = \{ \text{CS102, MA101} \}$ -Courses that were offered in BOTH semesters.
- $A - B = \{ \text{CS101} \}$ -Courses that existed in Fall but disappeared in Spring



To retain all duplicates, use the

- **union all**- it also shows the duplicate rows.
- *SELECT column_name(s) FROM table1
UNION ALL
SELECT column_name(s) FROM table2;*
- {CS101, CS102, MA101, CS101, CS102, EC201, MA101, MA101 }
- **intersect all**- returns common records with duplicates, but only as many as appear in both (minimum count).
- { CS101, CS101, MA101, MA101 }
- **except all**. subtract B from A including duplicates.
- { CS102, MA101 }



Example

Counts in A:

•CS101 → 2

•CS102 → 1

•MA101 → 3

Counts in B:

•CS101 → 3

•MA101 → 2

•EC201 → 1

CS101:

A has 2

B has 3

$$\Rightarrow 2 - 3 = -1 \rightarrow 0$$

So CS101 appears **0 times**

CS102:

A has 1

B has 0

$$\Rightarrow 1 - 0 = 1$$

So CS102 appears **1 time**

MA101:

A has 3

B has 2

$$\Rightarrow 3 - 2 = 1$$

So MA101 appears **1 time**

Final EXCEPT ALL result:

{ CS102, MA101 }

- **CS101:**
- appears **2 times** in A
- appears **3 times** in B
 $\Rightarrow \min(2,3) = 2$
So CS101 appears **2 times** in result
- **MA101:**
- appears 3 times in A
- appears 2 times in B
 $\Rightarrow \min(3,2) = 2$
So MA101 appears **2 times** in result
- **Final INTERSECT ALL result:**
{ CS101, CS101, MA101, MA101 }



Null Values

- It is possible for tuples to have a null value, denoted by **null**, for some of their attributes
- null signifies an unknown value or that a value does not exist.
- The result of any arithmetic expression involving **null** is **null**
 - **Example: $5 + \text{null}$ returns null**
- **The predicate *is null* can be used to check for null values.**
 - Example: Find all instructors whose salary is null.

select name
from instructor
where salary is null

- The predicate **is not null** succeeds if the value on which it is applied is not null.



Null Values (Cont.)

- NULL = **missing / unknown / not available** (not zero, not empty).

SQL treats as **unknown** the result of any comparison involving a null value (other than predicates **is null** and **is not null**).

- **Example: $5 < \text{null}$ or $\text{null} \neq \text{null}$ or $\text{null} = \text{null}$**

- Boolean = **True / False / Unknown**

- The predicate in a **where** clause can involve Boolean operations (**and**, **or**, **not**); thus the definitions of the Boolean operations need to be extended to deal with the value **unknown**.

- **and** : $(\text{true and unknown}) = \text{unknown}$,
 $(\text{false and unknown}) = \text{false}$,
 $(\text{unknown and unknown}) = \text{unknown}$
- **or**: $(\text{unknown or true}) = \text{true}$,
 $(\text{unknown or false}) = \text{unknown}$
 $(\text{unknown or unknown}) = \text{unknown}$

**select roll
from student
where marks > 50;**

- Result of **where** clause predicate is treated as *false* if it evaluates to *unknown*



Aggregate Functions

- These functions operate on the multiset of values of a column of a relation, and return a value
- **COUNT()**: Returns the number of rows or non-NULL values in a set.
- **SUM()**: Returns the total sum of a numeric column.
- **AVG()**: Returns the average value of a numeric column.
- **MIN()**: Returns the smallest value within a column.
- **MAX()**: Returns the largest value within a column.



Aggregate Functions Examples

- Find the average salary of instructors in the Computer Science department
 - **select** **avg** (*salary*)
from *instructor*
where *dept_name*= 'Comp. Sci.';
- Find the total number of instructors who teach a course in the Spring 2018 semester
 - **select** **count** (**distinct** *ID*)
from *teaches*
where *semester* = 'Spring' and *year* = 2018;
- Find the number of tuples in the *course* relation
 - **select** **count** (*)
from *course*;



Aggregate Functions – Group By

- Aggregate functions perform a calculation on a set of values (a group of rows) and return a single value for each group
- Find the average salary of instructors in each department
 - **select dept_name, avg (salary) as avg_salary**
from instructor
group by dept_name;

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

<i>dept_name</i>	<i>avg_salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000



Aggregation (Cont.)

- What is the average salary of each department?
- Attributes in **select** clause outside of aggregate functions must appear in **group by** list
 - */* erroneous query */*
select *dept_name, ID, avg (salary)*
from *instructor*
group by *dept_name;*

In **SELECT**, every column must be either:
1. inside an aggregate function (avg, sum, count, max, min)
OR
2. present in **GROUP BY**

<i>dept_name</i>	<i>avg_salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000



Illegal Queries Using Group Functions

Any column or expression in the SELECT list that is not an aggregate function must be in the GROUP BY clause.

```
SQL> SELECT      deptno, COUNT(ename)
      2  FROM      emp;
```

```
SELECT deptno, COUNT(ename)
      *
ERROR at line 1:
ORA-00937: not a single-group group function
```

Column missing in the GROUP BY clause



Illegal Queries

Using Group Functions

- .You cannot use the WHERE clause to restrict groups.
- .You use the HAVING clause to restrict groups.

```
SQL> SELECT      deptno, AVG(sal)
  2  FROM          emp
  3  WHERE         AVG(sal) > 2000
  4  GROUP BY     deptno;
```

```
WHERE AVG(sal) > 2000
      *
```

```
ERROR at line 3:
```

```
ORA-00934: group function is not allowed here
```

Cannot use the WHERE clause
to restrict groups



Aggregate Functions – Having Clause

- Find the names and average salaries of all departments whose average salary is greater than 42000

```
select dept_name, avg (salary) as avg_salary  
from instructor  
group by dept_name  
having avg (salary) > 42000;
```

- Note: predicates in the **having** clause are applied after the formation of groups whereas predicates in the **where** clause are applied before forming groups



Nested Subqueries

- SQL provides a mechanism for the nesting of subqueries. A **subquery** is a **select-from-where** expression that is nested within another query.
- The nesting can be done in the following SQL query

```
select  $A_1, A_2, \dots, A_n$   
from  $r_1, r_2, \dots, r_m$   
where  $P$ 
```

as follows:

- **From clause:** r_i can be replaced by any valid subquery
- **Where clause:** P can be replaced with an expression of the form:

B <operation> (subquery)

B is an attribute and <operation> to be defined later.

- **Select clause:**
 A_i can be replaced by a subquery that generates a single value.



■ Nested Query

- a (inner) query can be nested within the **WHERE-clause of another (outer) query**
- apply the outer query to the result of the inner query
- can have several levels of nested queries
- comparison operators are used to compare values of union compatible tuples
- For example: IN; >, <, =, <>; ALL, SOME, ANY



Set Membership



Set Membership Operators

Set membership operations determine whether **a value or a row exists** within a specified set of values or the result set of another query.

```
SELECT <column(s)>  
FROM table  
WHERE <condn operator>  
(SELECT <column>  
FROM table);
```

These operators determine whether a specific element is included within a set

Some queries requires that existing values in the database be fetched and then used in a comparison condition. Such queries can be conveniently formulated by using nested queries, which are complete **select-from-where blocks** within the **WHERE** clause of another query.



Using IN and NOT IN

The IN and NOT IN operators are the most common ways to check if a value belongs to a set of values or the results of a subquery.

■ select customers from the USA

- **SELECT first_name, country**
- **FROM Customers**
- **WHERE country IN ('USA');**

select rows if the country is either USA or UK

- **SELECT first_name, country**
- **FROM Customers**
- **WHERE country IN ('USA', 'UK');**

■ select rows where country is not in UK or UAE

- **SELECT first_name, country**
- **FROM Customers**
- **WHERE country NOT IN ('UK', 'UAE');**



Using IN and NOT IN

- Find **unique customer names (cname)** from the **Borrower** table who have an account in the bank branch **'SFU'**.

SELECT DISTINCT cname

FROM **Borrower**

WHERE cname IN

(**SELECT** cname **FROM** **Account** **WHERE**
bname = 'SFU')

- This subquery returns the list of all customers who have an account in branch **SFU**.
- Customers who borrowed money AND also have an account in SFU branch.**



Set Membership

- Find courses offered in Fall 2017 and in Spring 2018 (INTERSECT concept using IN)
- Find courses offered in Fall 2017 but not in Spring 2018 (MINUS concept using NOT IN)
- Name all instructors whose name is **neither “Mozart” nor Einstein”**
- Find the total number of (distinct) students who have taken course sections taught by the instructor with *ID* 10101

(Count how many different students took any course section taught by instructor 10101.)



- *Find courses offered in Fall 2017 and in Spring 2018:*

```
select distinct course_id
from section
where semester = 'Fall' and year= 2017 and
       course_id in ( select course_id
                        from section
                        where semester = 'Spring' and year= 2018);
```

- *Find courses offered in Fall 2017 but not in Spring 2018*

```
select distinct course_id
from section
where semester = 'Fall' and year= 2017 and
       course_id not in (select course_id
                             from section
                             where semester = 'Spring' and year= 2018);
```



Set Membership (Cont.)

- Name all instructors whose name is neither “Mozart” nor Einstein”

```
select distinct name  
from instructor  
where name not in ('Mozart', 'Einstein')
```

- Find the total number of (distinct) students who have taken course sections taught by the instructor with *ID* 10101

```
select count (distinct ID)  
from takes where (course_id, sec_id, semester, year) in  
    (select course_id, sec_id, semester, year  
     from teaches  
     where teaches.ID = 10101);
```

- Note: Above query can be written in a much simpler manner.
The formulation above is simply to illustrate SQL features

- Subquery finds all sections taught by instructor 10101
- Main query counts distinct students who took any of those sections



Set Comparison



Set Comparison operators

- The SQL **SOME** clause (interchangeable with **ANY**) is a set comparison operator that evaluates a condition against a set of values returned by a subquery and returns **TRUE** if the condition is satisfied by at least one value in that set.
 - **SELECT** column_name
 - **FROM** table_name
 - **WHERE** expression comparison_operator **SOME** (subquery);
- Find the names of employees who work in any department located in 'London'.
 - **SELECT** employee_name
 - **FROM** employees
 - **WHERE** department_id = **SOME** (
 - **SELECT** department_id
 - **FROM** departments
 - **WHERE** department_location = 'London'
-);
- Find names of instructors with salary greater than that of some (at least one) instructor in the Biology department.



Set Comparison – “some” Clause

- Find names of instructors with salary greater than that of some (at least one) instructor in the Biology department.

```
select distinct T.name  
from instructor as T, instructor as S  
where T.salary > S.salary and S.dept name = 'Biology';
```

- Same query using > **some** clause

```
select name  
from instructor  
where salary > some (select salary  
                        from instructor  
                        where dept name = 'Biology');
```

The instructors with salary > (salary of some instructor in the '**Biology**' department) get returned.

> SOME (Greater than any)

This returns true if the value on the left side is greater than at least one value in the subquery result set. This is equivalent to being greater than the *minimum* value returned by the subquery.



Definition of “some” Clause

- $F <comp> \text{some } r \Leftrightarrow \exists t \in r \text{ such that } (F <comp> t)$
- Where $<comp>$ can be: $<, \leq, >, =, \neq$

SOME: if **any one** is true $\Rightarrow$
overall TRUE

SOME =
“at least
one”

“If there
exists one
value
bigger than
5 (here 6),
then
it becomes
true.”

$$(5 < \text{some} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{true}$$

(read: 5 < some tuple in the relation)

$$(5 < \text{some} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{false}$$

F is compared with the values
returned by the subquery, and the
condition becomes TRUE if it
matches at least one value.

$$(5 = \text{some} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true}$$

There is **at least one** element t in
result set r which makes the
comparison true.

$$(5 \neq \text{some} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true (since } 0 \neq 5)$$

(= some) $\equiv$ in
However, ($\neq$ some) $\not\equiv$ not in



Set Comparison – “all” Clause

- The ANY and ALL operators allow you to perform a comparison between a single column value and a range of other values.
- ALL means that the condition **will be true only if the operation is true for all values in the range.**
- ANY means that the condition will be **true if the operation is true for any of the values in the range.**
- Find the names of all instructors whose salary is greater than the salary of all instructors in the Biology department.

```
select name  
from instructor  
where salary > all (select salary  
                        from instructor  
                        where dept name = 'Biology');
```



Definition of “all” Clause

- $F \text{ <comp> all } r \Leftrightarrow \forall t \in r (F \text{ <comp> } t)$ **ALL behaves like AND**
- F must satisfy the comparison with each element in r”

$$(5 < \text{all } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{false}$$

ALL: only if **all are true** $\Rightarrow$ overall TRUE

$$(5 < \text{all } \begin{array}{|c|} \hline 6 \\ \hline 10 \\ \hline \end{array}) = \text{true}$$

$$(5 = \text{all } \begin{array}{|c|} \hline 4 \\ \hline 5 \\ \hline \end{array}) = \text{false}$$

$$(5 \neq \text{all } \begin{array}{|c|} \hline 4 \\ \hline 6 \\ \hline \end{array}) = \text{true (since } 5 \neq 4 \text{ and } 5 \neq 6)$$

$(\neq \text{all}) \equiv \text{not in}$

However, $(= \text{all}) \not\equiv \text{in}$



Test for Empty Relations

- The **exists** construct returns the value **true** if the argument subquery is nonempty.
- **exists** $r \Leftrightarrow r \neq \emptyset$
- EXISTS r evaluates to **true** if the subquery r returns at least one row (is nonempty)
- **not exists** $r \Leftrightarrow r = \emptyset$
- NOT EXISTS r evaluates to **true** if the subquery r returns no rows (is empty)
- EXISTS checks **whether the subquery returns at least ONE row.**
- If subquery returns **at least one row** → **TRUE**
- If subquery returns **0 rows** → **FALSE**



EXISTS and NOT EXISTS

- The EXISTS operator is used to test for the existence of any record in a subquery.
- *ELECT column_name(s)*
FROM table_name
WHERE
- **EXISTS**
(SELECT column_name FROM table_name WHERE condition);
- The NOT EXISTS operator is the exact opposite. It checks if a subquery returns any records.
- EXISTS is used to check whether the result of a correlated nested query is empty (contains no tuples) or not.
- The result of EXISTS is a Boolean value TRUE or FALSE.



Correlated Nested Query

- If a condition in the WHERE-clause of the inner query references some attributes of a relation declared in the outer query, **the two queries are said to be correlated.**
- **The nested query is evaluated once for each tuple in the outer query.**
- The result of a correlated nested query is *different for each tuple (or combination of tuples) of the relation(s) the outer query*



EXISTS and NOT EXISTS

Customers

customer_id	customer_name
1	Ravi
2	Meena
3	John

Orders

order_id	customer_id
101	1
102	1
103	3

```
SELECT customer_id,  
customer_name  
FROM Customers  
WHERE customer_id IN (  
    SELECT customer_id  
    FROM Orders  
);
```

```
SELECT 1 FROM Orders WHERE  
Orders.customer_id = 1;
```

If TRUE → that customer is included in output

1. Displays the **customer_id** and **customer_name** of customers **who have placed at least one order**.
2. List customers who have NOT placed any order.



EXISTS and NOT EXISTS

- EXISTS checks whether the subquery returns **at least one row**.
- Displays the **customer_id** and **customer_name** of customers **who have placed at least one order**.
 - **SELECT** customer_id, customer_name
 - **FROM** Customers
 - **WHERE EXISTS (**
 - **SELECT 1**
 - **FROM** Orders
 - **WHERE Customers.customer_id = Orders.customer_id);**
- **SELECT DISTINCT**
C.customer_id,
C.customer_name
FROM Customers C
JOIN Orders O
ON C.customer_id =
O.customer_id;
- This is a **correlated subquery** -> Customers.customer_id
- The subquery is executed **for each customer**.
- EXISTS returns customers for whom at least one matching record exists in Orders
- In EXISTS, we do not care *what* is returned, only whether a row exists.



EXISTS and NOT EXISTS

- List customers who have NOT placed any order.
- Only those customers whose ID does not appear in Orders table
- SELECT customer_id, customer_name FROM Customers C
- WHERE NOT EXISTS
- (SELECT 1 FROM Orders O WHERE O.customer_id = C.customer_id);
- Meena (2) → no order



EXISTS and NOT EXISTS

- Find all courses taught in **both the Fall 2017 semester and in the Spring 2018 semester**”
- Find all students who have taken all courses offered in the Biology department.



Use of “exists” Clause

- “Find all courses taught in both the Fall 2017 semester and in the Spring 2018 semester”

```
select course_id
from section as S
where semester = 'Fall' and year = 2017 and
      exists (select *
              from section as T
              where semester = 'Spring' and year = 2018
                 and S.course_id = T.course_id);
```

Take all courses in Fall 2017

- For each Fall 2017 course S.course_id, check:
- Is there any row in Spring 2018 where course_id='CS101'?
- “Does this same course_id appear in Spring 2018 also?”
- If at least one Spring 2018 row exists with same course_id → TRUE
- **Correlation name** – variable S in the outer query
- **Correlated subquery** – the inner query



Use of “not exists” Clause

- Find all students who have taken all courses offered in the Biology department.

```
select distinct S.ID, S.name  
from student as S  
where not exists ( (select course_id  
                    from course  
                    where dept_name = 'Biology')  
except  
                    (select T.course_id  
                    from takes as T  
                    where S.ID = T.ID));
```

- First nested query lists all courses offered in Biology
- Second nested query lists all courses a particular student took
- Note that $X - Y = \emptyset \Leftrightarrow X \subseteq Y$
- Note: Cannot write this query using = all and its variants



Test for Absence of Duplicate Tuples

- The **unique** construct tests whether a subquery has any duplicate tuples in its result.
- It returns **TRUE** if the subquery output contains **NO duplicate rows**
The **unique** construct evaluates to “true” if a given subquery contains no duplicates .
- **Find all courses that were offered at most once in 2017**

SECTION table (2017)

course_id	sec_id	year
CS101	1	2017
CS101	2	2017
CS102	1	2017
CS103	1	2016



Test for Absence of Duplicate Tuples

- Find all courses that were offered at most once in 2017

```
select T.course_id
from course as T
where unique ( select R.course_id
               from section as R
               where T.course_id= R.course_id
                 and R.year = 2017);
```

It collects all rows in SECTION where that course is offered in the year 2017

- This subquery runs **for every course T**.
- It is a **correlated subquery** because it uses T.course_id.
- If the course is offered **twice in 2017**, section table will have **two rows**, both returning the *same* course_id.
- This query works because: if a course appears more than once in section table in 2017, the subquery repeats course_id → UNIQUE fails → course removed



Subqueries in the From Clause



Subqueries in the From Clause

- A subquery inside the FROM clause acts like a temporary table (virtual table).
This result is NOT stored permanently. SQL temporarily keeps it and passes it to outer query.
- SQL allows a subquery expression to be used in the **from** clause

- Find the average instructors' salaries of those departments where the average salary is greater than \$42,000."

```
select dept_name, avg_salary  
from ( select dept_name, avg (salary) as avg_salary  
      from instructor  
      group by dept_name )  
where avg_salary > 42000;
```

- Note that we do not need to use the **having** clause

```
select dept_name,  
avg_salary  
from (temp_table)  
where avg_salary >  
42000;
```




Subqueries in the Form Clause

- Another way to write above query

```
select dept_name, avg_salary
from ( select dept_name, avg (salary)
      from instructor
      group by dept_name)
      as dept_avg (dept_name, avg_salary)
where avg_salary > 42000;
```

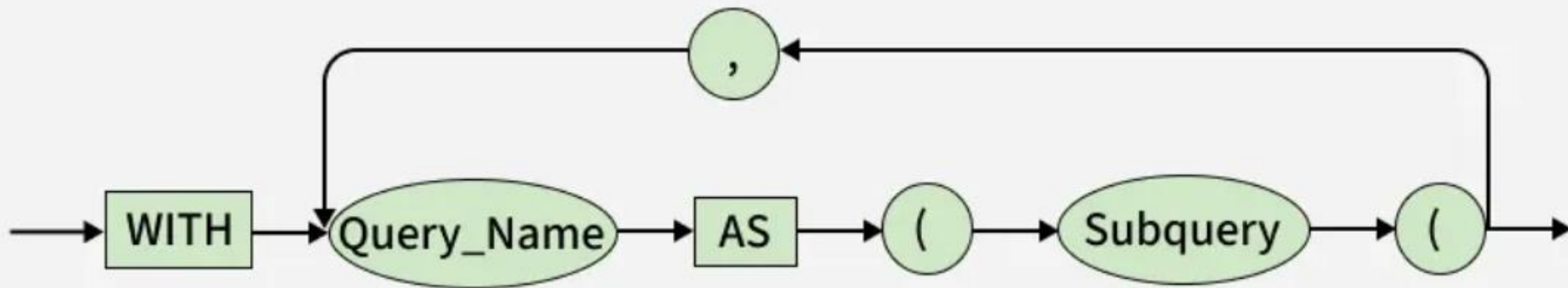
- This is just like giving a name to the temporary table and its columns.

dept_name	avg_salary
CSE	48000
ECE	41000
MECH	46000



With Clause(Common Table Expression or CTE)

The **with** clause provides a way of defining a temporary relation whose definition is available **only to the query in which the with clause occurs**.



EmployeeID	Name	Department	Salary
1	Martin	HR	65000
2	Bob	Sales	75000
3	Martin	HR	65000
4	Charlie	Marketing	88000
5	Diana	Sales	75000



With Clause(Common Table Expression or CTE)

- **Finding Employees with Above-Average Salary:** Find all employees whose salary is higher than the average salary of all employees in the database.
- The query calculates the average salary using the WITH clause and compares each employee's salary against this average to return those with above-average salaries.

- **WITH AvgSalaryCTE (averageValue) AS**

**(
SELECT AVG(Salary)
FROM Employees
)**

- **SELECT EmployeeID, Name, Salary**

FROM Employees

WHERE Salary > (SELECT averageValue FROM AvgSalaryCTE);

This query returns employees whose salary is higher than the average salary.



With Clause(Common Table Expression or CTE)

- Find all departments with the maximum budget

with *max_budget (value)* **as**
(select *max(budget)*
from *department***)**

- *max_budget* is a temporary relation with one column called *value*.

select *department.name*
from *department*, *max_budget*
where *department.budget = max_budget.value;*

- First find the **highest budget** among all departments
- Then print department name(s) that have that same highest budget



Complex Queries using With Clause

- Find all departments where the total salary is greater than the average of the total salary at all departments

1. Find **total salary per department**
2. Find **average of these totals**
3. Return departments where **dept total > average total**



Complex Queries using With Clause

- Find all departments where the total salary is greater than the average of the total salary at all departments

```
with dept_total (dept_name, value) as  
  (select dept_name, sum(salary)  
   from instructor  
   group by dept_name),
```

```
dept_total_avg(value) as  
  (select avg(value)  
   from dept_total)
```

```
select dept_name  
from dept_total, dept_total_avg  
where dept_total.value > dept_total_avg.value;
```

dept_total becomes a temporary table with columns (dept_name, value). This gives total salary per department.

It computes average of totals (not average of individual instructor salaries). dept_total_avg is a 1-row table containing the average of department totals.



Complex Queries using With Clause

Find all departments where the total salary is greater than the average of the total salary at all departments

dept_name	value (sum salary)	value (avg)
CSE	180000	130000
ECE	120000	
MECH	90000	

dept_name
CSE



Scalar Subquery

- A **scalar subquery** is a subquery that returns **exactly ONE value** (one row, one column).
Scalar subquery is one which is used where a single value is expected-Returns one value only
- If it returns **more than one row** → **error**
- Aggregate functions like COUNT(), SUM(), AVG() naturally return a **single value**, so they are commonly used in scalar subqueries
- It is used where SQL expects a single value, such as in the SELECT or WHERE clause.
- **List all departments along with the number of instructors in each department**



- **List all departments along with the number of instructors in each department**

```
select dept_name,  
       ( select count(*)  
         from instructor  
         where department.dept_name =  
instructor.dept_name )  
       as num_instructors/AS instructor_count  
from department;
```

- The subquery returns **one number per department**
- For every department (like CSE, ECE...), it runs the subquery
- The subquery counts instructors **only for that department**



Example

```
SELECT dept_name,  
      (SELECT name  
       FROM instructor  
       WHERE department.dept_name =  
instructor.dept_name)  
FROM department;
```

- Subquery returns multiple rows → not scalar →
O/p?



Modification of the Database

Database modification means changing the data stored in tables.

3 basic ways to change data inside a table:

- Deletion of tuples from a given relation.
- Insertion of new tuples into a given relation
- Updating of values in some tuples in a given relation



Deletion

1. Delete all instructors
2. Delete all instructors from the Finance department
3. *Delete all tuples in the instructor relation for those instructors associated with a department located in the Watson building.*
4. **Delete all instructors whose salary is less than the average salary of instructors**



Deletion

- Delete all instructors

delete from *instructor*

- Delete all instructors from the Finance department

delete from *instructor*

where *dept_name* = 'Finance';

- *Delete all tuples in the instructor relation for those instructors associated with a department located in the Watson building.*

delete from *instructor*

where *dept name* in (**select** *dept name*
from *department*
where *building* = 'Watson');



Deletion (Cont.)

- **Delete all instructors whose salary is less than the average salary of instructors**

delete from *instructor*

where *salary* <

(*select avg (salary)*

***from instructor*);**

SQL computes the **average salary**

once

Each tuple in *instructor* is compared with



- **Problem:** as we delete tuples from *instructor*, the average salary changes. If the average were recomputed after each deletion, the result would be **incorrect or inconsistent**.
- The apparent problem is that deleting tuples changes the table, which could change the average salary.
- Solution used in SQL:
 1. First, compute **avg** (salary) and find all tuples to delete- Compute AVG(salary) **once**
 2. Identify all tuples (rows) with salary less than that average
 3. Next, delete all tuples found above (without recomputing **avg** or retesting the tuples



Using With Clause

```
WITH avg_sal AS (  
    SELECT AVG(salary) AS avg_salary  
    FROM instructor  
)  
DELETE FROM instructor  
WHERE salary < (SELECT avg_salary FROM  
avg_sal);
```




Insertion

- Add a new tuple to *course*

insert into *course*

values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

Values are inserted **in the order of the table attributes**

- or equivalently

insert into *course* (*course_id*, *title*, *dept_name*, *credits*)

values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

Column names are **explicitly specified**

- Add a new tuple to *student* with *tot_creds* set to null

insert into *student*

values ('3003', 'Green', 'Finance', *null*); --NULL means **no value available**



Insertion (Cont.)

Convert eligible Music students into instructors

- Make each student **in the Music department who has earned more than 144 credit hours** an instructor in the Music department with a salary of \$18,000.

insert into *instructor*

```
select ID, name, dept_name, 18000  
from student  
where dept_name = 'Music' and total_cred > 144;
```

- SQL first **collects all rows from SELECT into a temporary result set**, then inserts them.-Inserts them **all at once** into instructor
- In an INSERT–SELECT statement, SQL first evaluates the SELECT query completely and stores the result in a temporary relation. Only after this step are the tuples inserted into the target table.

Otherwise queries like

insert into table1 select * from table1 -“self-copy”

would cause problem



Updates

- Give a **5% salary raise** to all instructors

```
update instructor  
set salary = salary * 1.05
```

- Give a 5% salary raise to those instructors **who earn less than 70000**

```
update instructor  
set salary = salary * 1.05  
where salary < 70000;
```

- Give a 5% salary raise to instructors **whose salary is less than average**

```
update instructor  
set salary = salary * 1.05  
where salary < (select avg (salary)  
from instructor);
```



Updates (Cont.)

- Increase salaries of instructors whose salary is **over \$100,000 by 3%, and all others by a 5%**
 - Write two **update** statements:
 - First statement updates **high-salary instructors**

```
update instructor  
  set salary = salary * 1.03  
  where salary > 100000;
```

Second statement updates **remaining instructors**

```
update instructor  
  set salary = salary * 1.05  
  where salary <= 100000;
```

- The order is important, conditions do not overlap
- Can be done better using the case statement (next slide)



Case Statement for Conditional Updates

- Requires **multiple statements**
- Error-prone if conditions overlap
- **Same query as before but with case statement**
- CASE works like **if–else** inside SQL. CASE allows conditional logic inside a single SQL statement.

update instructor

set salary = case

when salary <= 100000 then salary * 1.05

else salary * 1.03

end;



Example2

- **Recompute total credits** for each student by adding credits of all courses the student has **passed**.

- For each student S
 1. Find all courses taken by that student
 2. Exclude failed (F) and null grades
 3. Add up the course credits



Updates with Scalar Subqueries

- **Recompute and update tot_creds value for all students**
- Add credits of all courses the student has passed

UPDATE student S

SET tot_cred = (

SELECT SUM(credits)

FROM takes, course

WHERE takes.course_id = course.course_id

AND S.ID = takes.ID

AND takes.grade <> 'F'

AND takes.grade IS NOT NULL);

It computes the **total number of credits earned by one student S**, counting **only the courses the student has passed**

This query is usually inside an UPDATE statement, so it is executed **once per student**.

If no rows → sum is NULL → replace with 0

- Sets **tot_creds to null** for students who have not taken any course
- Instead of **sum(credits)**, use: **case**
when sum(credits) is not null then sum(credits) else 0 end



Improved solution using **CASE**: CASE is used to handle NULL values

UPDATE student S

SET **tot_cred** =

```
(  SELECT CASE
      WHEN SUM(course.credits) IS NOT NULL
      THEN SUM(course.credits)
      ELSE 0
    END
  FROM takes JOIN course
    ON takes.course_id = course.course_id
 WHERE takes.ID = S.ID
    AND takes.grade <> 'F'
    AND takes.grade IS NOT NULL
);
```

The query joins the takes and course tables, selects the passed courses of a particular student, and computes the total earned credits using the SUM aggregate function.



End of Chapter 3